

Lec7 CClass

- 声明: declaration
- 实现: definition, implementation
- 调用: call, invoke

Class

Public&private

是否能从外部访问、

Default member functions

- compiler will provided by default.
 - constructor
 - copy constructor
 - assignment operator 赋值运算符
 - move constructor
 - move assignment operator
 - destructor

constructor

- no return type
- invoked only when an object is created 只有在创建对象时才调用构造函数

```
Point2D p(2,3);  
p.Point2D(3,5);  ← error, 不能调用构造函数来reset一个对象  
p = Point2D(3,5) //ok, 用构造函数来覆盖一个现有对象, 也就是调用  
assignment operator
```

C++

default constructor

- a constructor without arguments
- 如果没有declare任何constructor, compiler给一个default constructor
 - 这个default constructor相当于:

C++

```
class Point{
public:
    int a;
    int b;
    Point();      ← default constructor
}
Point::Point(){} ← default constructor
// 这里a,b都是未定义值
```

- 如果提供了一个构造函数

C++

```
class Point2D{
public:
    int x;
    int y;
    Point2D(int x, int y);      ← 无初始值
};
Point2D::Point2D(int x, int y){
    this->x = x;
    this->y = y;
}
```

也可以合并写成

```
class Point2D{
public:
    int x;
    int y;
    Point2D(int x = 0, int y = 0); ← 有初始值，也算default
    constructor
};
```

```
int main(){
    Point2D p; ←error, 需要提供argument, default constructor已经
    不存在
}
```

- default & delete

C++

```
Point2D() = default; ← 让编译器生成一个default constructor
等价于
Point2D::Point2D(){};
```

C++

```
Point2D() = delete; ← 禁止使用default constructor
```

- 会引起ambiguous的example

```
class Point2D{  
public:  
    Point2D() = default;  
    Point2D(int x = 0, int y = 0); ← 带初始值  
}
```

C++

call `Point2D p;` 的时候, 会ambiguous

带初始值的function可以不给/给少parameter

Copy Constructor & assignment operator

Copy Constructor

```
Point2D(const Point& p) :  
    x(p.x), y(p.y){};
```

C++

Assignment Operator

```
Point2D& operator= (const Point2D& p){ ← 返回一个Point2D的ref, 传入p地址  
    this->x = p.x;  
    this->y = p.y;  
    return *this; ← 返回当前对象的引用  
}
```

C++

- this是的类型是Point2D* const, 指向当前对象的指针
 - `*this`: 当前对象本身, 类型是Point2D
- 区分什么时候调用的是哪一个

```
Point2D p2 = p1; ← copy constructor  
void foo(Point2D p); foo(p2); ← copy constructor  
p2 = Point(2,3); ← Assignment operator
```

C++

Destructor

```
~Point2D(){} 
```

C++

Initialization

- 区分assignment和initialization: {}里面是赋值。eg.{this->x = x;}// () 是直接初始化。:
x(x), y(y)
 - 因为有一部分成员不能先构造后赋值
 - const
 - reference成员
 - 没有default constructor的

```
class A {  
public:  
    A (int x) : x(x){}; ←带initialization的constructor  
    int x;  
}
```

C++

几种constructor的区分

1. `A(): A(0,0){};` 算是default constructor
 1. 没有参数
 2. `: A(0,0)` delegating constructor(委托构造), 实际上调用了有参的constructor
2. `A(int x, int y): x(x),y(y){};` Non-default constructor
3. `A(const A& a) : x(a.x),y(a.y){};` copy constructor
4. Converting constructor
 1. 编译器进行的implicit转换

```
1. Student(const string& name, int id=0);  
   string name;  
   string name = "Alice";  
   Student s1 = name;      ←合法, 把string→ Student. 实际上  
                           创建了一个Student Object  
   Student s2 = "Alice";   ← error. 发生了两步implicit  
                           converting  
   Student s3 = string("Alice"); ←ok  
  
   explicit Student(const string& name, int id=0){}
```

C++

禁止使用implicit converting

accessing data